EC200D&ECx00G&EC600U&EG800G Series

QuecOpen(SDK) Voice API Reference Manual

LTE Standard Module Series

Version: 1.0

Date: 2023-09-23

Status: Released

At Quectel, our aim is to provide timely and comprehensive services to our customers. If you require any assistance, please contact our headquarters:

Quectel Wireless Solutions Co., Ltd.
Building 5, Shanghai Business Park Phase III (Area B), No.1016 Tianlin Road, Minhang District, Shanghai 200233, China
Tel: +86 21 5108 6236
Email: info@quectel.com

Or our local offices. For more information, please visit:
http://www.quectel.com/support/sales.htm.

For technical support, or to report documentation errors, please visit:
http://www.quectel.com/support/technical.htm.
Or email us at: support@quectel.com.

Legal Notices

We offer information as a service to you. The provided information is based on your requirements and we make every effort to ensure its quality. You agree that you are responsible for using independent analysis and evaluation in designing intended products, and we provide reference designs for illustrative purposes only. Before using any hardware, software or service guided by this document, please read this notice carefully. Even though we employ commercially reasonable efforts to provide the best possible experience, you hereby acknowledge and agree that this document and related services hereunder are provided to you on an “as available” basis. We may revise or restate this document from time to time at our sole discretion without any prior notice to you.

Use and Disclosure Restrictions

License Agreements

Documents and information provided by us shall be kept confidential, unless specific permission is granted. They shall not be accessed or used for any purpose except as expressly provided herein.

Copyright

Our and third-party products hereunder may contain copyrighted material. Such copyrighted material shall not be copied, reproduced, distributed, merged, published, translated, or modified without prior written consent. We and the third party have exclusive rights over copyrighted material. No license shall be granted or conveyed under any patents, copyrights, trademarks, or service mark rights. To avoid ambiguities, purchasing in any form cannot be deemed as granting a license other than the normal non-exclusive, royalty-free license to use the material. We reserve the right to take legal action for noncompliance with abovementioned requirements, unauthorized use, or other illegal or malicious use of the material.

Trademarks

Except as otherwise set forth herein, nothing in this document shall be construed as conferring any rights to use any trademark, trade name or name, abbreviation, or counterfeit product thereof owned by Quectel or any third party in advertising, publicity, or other aspects.

Third-Party Rights

This document may refer to hardware, software and/or documentation owned by one or more third parties ("third-party materials"). Use of such third-party materials shall be governed by all restrictions and obligations applicable thereto.

We make no warranty or representation, either express or implied, regarding the third-party materials, including but not limited to any implied or statutory, warranties of merchantability or fitness for a particular purpose, quiet enjoyment, system integration, information accuracy, and non-infringement of any third-party intellectual property rights with regard to the licensed technology or use thereof. Nothing herein constitutes a representation or warranty by us to either develop, enhance, modify, distribute, market, sell, offer for sale, or otherwise maintain production of any our products or any other hardware, software, device, tool, information, or product. We moreover disclaim any and all warranties arising from the course of dealing or usage of trade.

Privacy Policy

To implement module functionality, certain device data are uploaded to Quectel's or third-party's servers, including carriers, chipset suppliers or customer-designated servers. Quectel, strictly abiding by the relevant laws and regulations, shall retain, use, disclose or otherwise process relevant data for the purpose of performing the service only or as permitted by applicable laws. Before data interaction with third parties, please be informed of their privacy and data security policy.

Disclaimer

a) We acknowledge no liability for any injury or damage arising from the reliance upon the information.
b) We shall bear no liability resulting from any inaccuracies or omissions, or from the use of the information contained herein.
c) While we have made every effort to ensure that the functions and features under development are free from errors, it is possible that they could contain errors, inaccuracies, and omissions. Unless otherwise provided by valid agreement, we make no warranties of any kind, either implied or express, and exclude all liability for any loss or damage suffered in connection with the use of features and functions under development, to the maximum extent permitted by law, regardless of whether such loss or damage may have been foreseeable.
d) We are not responsible for the accessibility, safety, accuracy, availability, legality, or completeness of information, advertising, commercial offers, products, services, and materials on third-party websites and third-party resources.

About the Document

Revision History

Version	Date	Author	Description
-	2023-08-22	Marvin NING	Creation of the document
1.0	2023-09-23	Marvin NING	First official release

Contents

Table Index

1 Introduction

Quectel EC200D series, ECx00G family, EC600U series, and EG800G-CN modules support QuecOpen® solution. QuecOpen® is an embedded development platform based on RTOS. It is intended to simplify the design and development of IoT applications. For more information on QuecOpen®, see ***documents [1]*** and ***[2]***.

This document is applicable to QuecOpen® solution based on CSDK build environment. It introduces the voice API and related examples on Quectel EC200D series, ECx00G family, EC600U series, and EG800G-CN modules in QuecOpen® solution.

1.1. Applicable Modules

Table 1: Applicable Modules

Module Family	Module
-	EC200D Series
-	EC600U Series
ECx00G	EC200G-CN
	EC600G-CN
	EC800G-CN
-	EG800G-CN

NOTE

For the EC200D series module, the file paths mentioned in this document do not contain *components*.

2 Voice API

2.1. Header File

ql_api_voice_call.h, the header file of voice API, is in the *\components\ql-kernel\inc* directory. Unless otherwise specified, all the header files mentioned in this document are in this directory.

2.2. API Overview

Table 2: API Overview

Function	Description
ql_voice_auto_answer()	Automatically answers a voice call.
ql_voice_call_start()	Starts a voice call.
ql_voice_call_answer()	Answers a voice call.
ql_voice_call_end()	Hangs up a voice call.
ql_voice_call_start_dtmf()	Sends the DTMF.
ql_voice_call_wait_set()	Sets call waiting.
ql_voice_call_wait_get()	Gets the status of call waiting.
ql_voice_call_fw()	Sets or queries the supplementary service of call forwarding.
ql_voice_call_hold()	Handles call hold and multi-party calls.
ql_voice_call_clcc()	Gets the detailed information of all current calls.
ql_voice_call_callback_register()	Sets the callback function for voice call events.

2.3. API Description

2.3.1. ql_voice_auto_answer

This function automatically answers a voice call.

- **Prototype**

```
ql_vc_errcode_e ql_voice_auto_answer(uint8_t nSim, uint8_t times);
```

- **Parameter**

nSim:
[In] (U)SIM card index. Set this parameter to 0 if only one (U)SIM interface is supported.
0 (U)SIM card 1
1 (U)SIM card 2

times:
[In] Set the times of rings before automatic answering. *0* means automatic answering is disabled. Default value: 0.

- **Return Value**

See ***Chapter 2.3.1.1*** for error codes.

2.3.1.1. ql_vc_errcode_e

The enumeration of voice call error codes:

```
typedef enum
{
    QL_VC_SUCCESS                   = 0,
    QL_VC_ERROR                     = 1 | (QL_COMPONENT_VOICE_CALL << 16),
    QL_VC_NOT_INIT_ERR,
    QL_VC_PARA_ERR,
    QL_VC_NO_MEMORY_ERR,
    QL_VC_NO_CALL_ERR,
    QL_VC_SEM_CREATE_ERR,
    QL_VC_SEM_TIMEOUT_ERR,
}ql_vc_errcode_e;
```

- **Member**

Member	Description
QL_VC_SUCCESS	Successful execution.
QL_VC_ERROR	Failed execution.
QL_VC_NOT_INIT_ERR	The voice call is not initialized.
QL_VC_PARA_ERR	Parameter(s) error.
QL_VC_NO_MEMORY_ERR	Fail to apply for memory.
QL_VC_NO_CALL_ERR	No calls currently.
QL_VC_SEM_CREATE_ERR	Fail to create a semaphore.
QL_VC_SEM_TIMEOUT_ERR	The semaphore timeout period has expired.

2.3.2. ql_voice_call_start

This function starts a voice call.

- **Prototype**

```
ql_vc_errcode_e ql_voice_call_start(uint8_t nSim, char* dial_num);
```

- **Parameter**

nSim:
[In] (U)SIM card index. Set this parameter to 0 if only one (U)SIM interface is supported.
0 (U)SIM card 1
1 (U)SIM card 2

dial_num:
[In] Phone number to be dialed.

- **Return Value**

See ***Chapter 2.3.1.1*** for error codes.

NOTE

The interface is an asynchronous interface. The return value *QL_VC_SUCCESS* only indicates that the interface is called successfully. The callback function registered by *ql_voice_call_callback_register()* will be used to notify whether the voice call is successful or not.

2.3.3. ql_voice_call_answer

This function answers a voice call but does not support automatic answering.

- **Prototype**

```
ql_vc_errcode_e ql_voice_call_answer(uint8_t nSim);
```

- **Parameter**

nSim:
[In] (U)SIM card index. Set this parameter to 0 if only one (U)SIM interface is supported.
0 (U)SIM card 1
1 (U)SIM card 2

- **Return Value**

See ***Chapter 2.3.1.1*** for error codes.

2.3.4. ql_voice_call_end

This function hangs up a voice call.

- **Prototype**

```
ql_vc_errcode_e ql_voice_call_end(uint8_t nSim);
```

- **Parameter**

nSim:
[In] (U)SIM card index. Set this parameter to 0 if only one (U)SIM interface is supported.
0 (U)SIM card 1
1 (U)SIM card 2

- **Return Value**

See ***Chapter 2.3.1.1*** for error codes.

2.3.5. ql_voice_call_start_dtmf

This function sends the DTMF.

- **Prototype**

```
ql_vc_errcode_e ql_voice_call_start_dtmf(uint8_t nSim, char *dtmf, uint16_t duration);
```

- **Parameter**

nSim:
[In] (U)SIM card index.
0 (U)SIM card 1
1 (U)SIM card 2

dtmf:
[In] DTMF characters. The maximum number of characters is 32. Valid character: 0–9, A, B, C, D, *, #.

duration:
[In] Duration. Range: 1–10. Default value: 1. Unit: 100 ms.

- **Return Value**

See ***Chapter 2.3.1.1*** for error codes.

2.3.6. ql_voice_call_wait_set

This function sets call waiting.

- **Prototype**

```
ql_vc_errcode_e ql_voice_call_wait_set(uint8_t nSim, uint8_t mode);
```

- **Parameter**

nSim:
[In] (U)SIM card index. Set this parameter to 0 if only one (U)SIM interface is supported.
0 (U)SIM card 1
1 (U)SIM card 2

mode:
[In] Disable or enable the call waiting.
0 Disable
1 Enable

- **Return Value**

See ***Chapter 2.3.1.1*** for error codes.

2.3.7. ql_voice_call_wait_get

This function gets the status of call waiting.

- **Prototype**

```
ql_vc_errcode_e ql_voice_call_wait_get(uint8_t nSim, uint8_t    *mode);
```

- **Parameter**

nSim:
[In] (U)SIM card index. Set this parameter to 0 if only one (U)SIM interface is supported.
0 (U)SIM card 1
1 (U)SIM card 2

mode:
[Out] Status of call waiting.
0 Call waiting is disabled.
1 Call waiting is enabled.

- **Return Value**

See ***Chapter 2.3.1.1*** for error codes.

2.3.8. ql_voice_call_fw

This function sets or queries the supplementary service of call forwarding.

- **Prototype**

```
ql_vc_errcode_e ql_voice_call_fw(uint8_t nSim, int reason, int fwmode, char* phone_num);
```

- **Parameter**

nSim:
[In] (U)SIM card index. Set this parameter to 0 if only one (U)SIM interface is supported.
0 (U)SIM card 1
1 (U)SIM card 2

reason:
[In] Conditions or reasons for call forwarding.

- *0* Unconditional call forwarding
- *1* Mobile busy
- *2* No reply
- *3* Unreachable
- *4* All call forwarding
- *5* All conditional call forwarding

fwmode:
[In] The operating mode for call forwarding.

- *0* Disable
- *1* Enable
- *2* Query the status
- *3* Register
- *4* Erase

phone_num:
[In/Out] Target phone number of call forwarding.

- **Return Value**

See ***Chapter 2.3.1.1*** for error codes.

2.3.9. ql_voice_call_hold

This function handles call hold and multi-party calls.

- **Prototype**

```
ql_vc_errcode_e ql_voice_call_hold(uint8_t nSim, uint8_t n);
```

- **Parameter**

nSim:
[In] (U)SIM card index. Set this parameter to 0 if only one (U)SIM interface is supported.

- *0* (U)SIM card 1
- *1* (U)SIM card 2

n:
[In] Control of multi-party calls.
0 Release all held or waiting calls
1 Release the ongoing call; answer a held or waiting call
1X Release the call with index X, which can be an ongoing, held or waiting call
2 Set all ongoing calls to held calls and accept the held or waiting call
2X Request to accept calls with index X and hold other calls

- **Return Value**

See ***Chapter 2.3.1.1*** for error codes.

2.3.10. ql_voice_call_clcc

This function gets the detailed information of all current calls.

- **Prototype**

```
ql_vc_errcode_e ql_voice_call_clcc(uint8_t nSim, uint8_t *total, ql_vc_info_s vc_info[QL_VC_MAX_NUM]);
```

- **Parameter**

nSim:
[In] (U)SIM card index. Set this parameter to 0 if only one (U)SIM interface is supported.
0 (U)SIM card 1
1 (U)SIM card 2

total:
[Out] The total number of current calls.

vc_info:
[Out] Detailed information of all calls, including call index, call status and phone number. See ***Chapter 2.3.10.1*** for details.

- **Return Value**

See ***Chapter 2.3.1.1*** for error codes.

2.3.10.1. ql_vc_info_s

The structure of the detailed information of calls:

```
typedef struct
 {
     uint8_t idx;
     uint8_t direction;
     uint8_t status;
     uint8_t multiparty;
     char number[23];
 }ql_vc_info_s;
```

- **Parameter**

Type	Parameter	Description
uint8_t	*idx*	Index. Range: 1–7.
uint8_t	*direction*	Calling party. *0* Mobile originated *1* Mobile terminated
uint8_t	*status*	The status of a voice call. *0* Active *1* Held 2 Dialing *3* Alerting *4* Incoming call *5* Call waiting *7* Disconnected *8* Shaking
uint8_t	*multiparty*	*0* Not in a multi-party call *1* In a multi-party call
char	*number*	Phone number.

2.3.11. ql_voice_call_callback_register

This function sets the user callback function to handle the events related to the voice call.

- **Prototype**

```
void ql_voice_call_callback_register(ql_vc_event_handler_t cb);
```

- **Parameter**

cb:
[In] The callback function for voice call events. See ***Chapter 2.3.11.1*** for details.

- **Return Value**

See ***Chapter 2.3.1.1*** for error codes.

2.3.11.1. ql_vc_event_handler_t

The definition of this callback function:

```
typedef void (*ql_vc_event_handler_t)(uint8_t sim, ql_vc_event_id_e event_id, void *ctx);
```

- **Parameter**

sim:
[In] (U)SIM card index. Set this parameter to 0 if only one (U)SIM interface is supported.
0 (U)SIM card 1
1 (U)SIM card 2

event_id:
[In] Voice call event. See ***Chapter 2.3.11.2*** for details.

ctx:
[In] The pointer to inputting parameters to the callback function.

- **Return Value**

None

2.3.11.2. ql_vc_event_id_e

The enumeration of the voice call events:

```
typedef enum
 {
      QL_VC_INIT_OK_IND =          1| (QL_COMPONENT_VOICE_CALL << 16),
      QL_VC_RING_IND,
      QL_VC_CONNECT_IND,
      QL_VC_NOCARRIER_IND,
      QL_VC_ERROR_IND,
      QL_VC_CCWA_IND,
```

```
    QL_VC_DIAL_IND,
    QL_VC_ALERT_IND,
    QL_VC_HOLD_IND,
    QL_VC_DISCONNECT_IND,
    QL_VC_AUDIO_IND
}ql_vc_event_id_e;
```

- **Member**

Member	Description
QL_VC_INIT_OK_IND	Successful initialization (Not supported).
QL_VC_RING_IND	A new incoming call.
QL_VC_CONNECT_IND	Connect a voice call.
QL_VC_NOCARRIER_IND	Disconnect a voice call.
QL_VC_ERROR_IND	Unknown error.
QL_VC_CCWA_IND	Call waiting.
QL_VC_DIAL_IND	Initiate a call.
QL_VC_ALERT_IND	The other party rings when the caller calls.
QL_VC_HOLD_IND	Hold a call.
QL_VC_DISCONNECT_IND	Disconnect a voice call.
QL_VC_AUDIO_IND	Audio delivery over the network.

NOTE

QL_VC_NOCARRIER_IND and *QL_VC_DISCONNECT_IND* events are sent simultaneously when the call hangs up.

3 Examples

This chapter introduces how to use the above APIs for voice call development and simple debugging in application.

3.1. Development

voice_call_demo.c, the example file of the voice call, provided in the QuecOpen CSDK codes of the module, is in the *\components\ql-application\voice_call* directory. *voice_call_demo.c* contains examples related to call dialing, call answering, call waiting, call forwarding and other features. The entry function is *ql_voice_call_app_init()*, which enables *voice_call_demo_task*, as shown in the figure below:

```
QlOSStatus ql_voice_call_app_init(void)
{
    QlOSStatus err = QL_OSI_SUCCESS;

    err = ql_rtos_task_create(&vc_task, 4096, APP_PRIORITY_NORMAL, "vc_task", voice_call_demo_task, NULL, 10);
    if(err != QL_OSI_SUCCESS)
    {
        QL_VC_LOG("voice_call_demo_task created failed, ret = 0x%x", err);
    }

    return err;
}
```

ql_voice_call_app_init() is not called by default. As shown in the figure below, uncomment the function before calling it.

```
#ifdef QL_APP_FEATURE_VOICE_CALL
    ql_voice_call_app_init();
#endif
```

3.2. Debugging

You can debug the voice feature of the module through Quectel LTE OPEN EVB.

You can view information of the example after capturing the AP logs with *cooltools* or *ArmLogel*. EC600U series module uses *cooltools* tool to capture logs, and see ***document [3]*** for log capture method. EC200D series, ECx00G family and EG800G-CN modules use *ArmLogel* tool to capture logs, and see ***documents [4]*** and ***[5]*** for log capture method.

After *voice_call_demo_task()* is enabled, the callback function will be registered and the module waits for the incoming call. When receiving an incoming call, the module will automatically answer the call after 10 seconds, and then hang up automatically in 10 seconds, as shown in the figure below.

```
QL_VC_LOG("wait call");
while(1){
    ql_event_wait(&vc_event, QL_WAIT_FOREVER);
    switch(vc_event.id)
    {
        case QL_VC_RING_IND:
            ql_rtos_task_sleep_s(10);//
            err = ql_voice_call_answer(nSim);
            if(err != QL_VC_SUCCESS){
                QL_VC_LOG("ql_voice_call_answer FAIL");
            }else{
                QL_VC_LOG("ql_voice_call_answer OK");
            }
            break;
        case QL_VC_CONNECT_IND:
            QL_VC_LOG("CONNECT");
            ql_rtos_task_sleep_s(10);
            err = ql_voice_call_end(nSim);
            if(err != QL_VC_SUCCESS){
                QL_VC_LOG("ql_voice_call_end FAIL");
            }else{
                QL_VC_LOG("ql_voice_call_end OK");
            }
            break;
        case QL_VC_NOCARRIER_IND:
            QL_VC_LOG("NOCARRIER");
            break;
        default:
            QL_VC_LOG("event id = 0x%x",vc_event.id);
            break;
    } « end switch vc_event.id »
} « end while 1 »
```

NOTE

If you need to test other features of a voice call, such as actively starting a call, setting call waiting and call forwarding, you can enable the corresponding macro definition in this example to test.

4 Appendix References

Table 3: Related Documents

Document Name
[1] Quectel_ECx00G&EC600U&EG800G_Series_QuecOpen(SDK)_Quick_Start_Guide
[2] Quectel_EC200D_Series_QuecOpen(SDK)_Quick_Start_Guide
[3] Quectel_EC600U_Series_QuecOpen(SDK)_Log_Capture_Guide
[4] Quectel_EC200D_Series_Log_Capture_Guide
[5] Quectel_ECx00G&EG800G_Series_QuecOpen(SDK)_Log_Capture_Guide

Table 4: Terms and Abbreviations

Abbreviation	Description
AP	Application Processor
API	Application Programming Interface
App	Application
DTMF	Dual Tone Multi Frequency
EVB	Evaluation Board
IoT	Internet of Things
LTE	Long-Term Evolution
PC	Personal Computer
RTOS	Real-Time Operating System
SDK	Software Development Kit
(U)SIM	(Universal) Subscriber Identity Module
USB	Universal Serial Bus